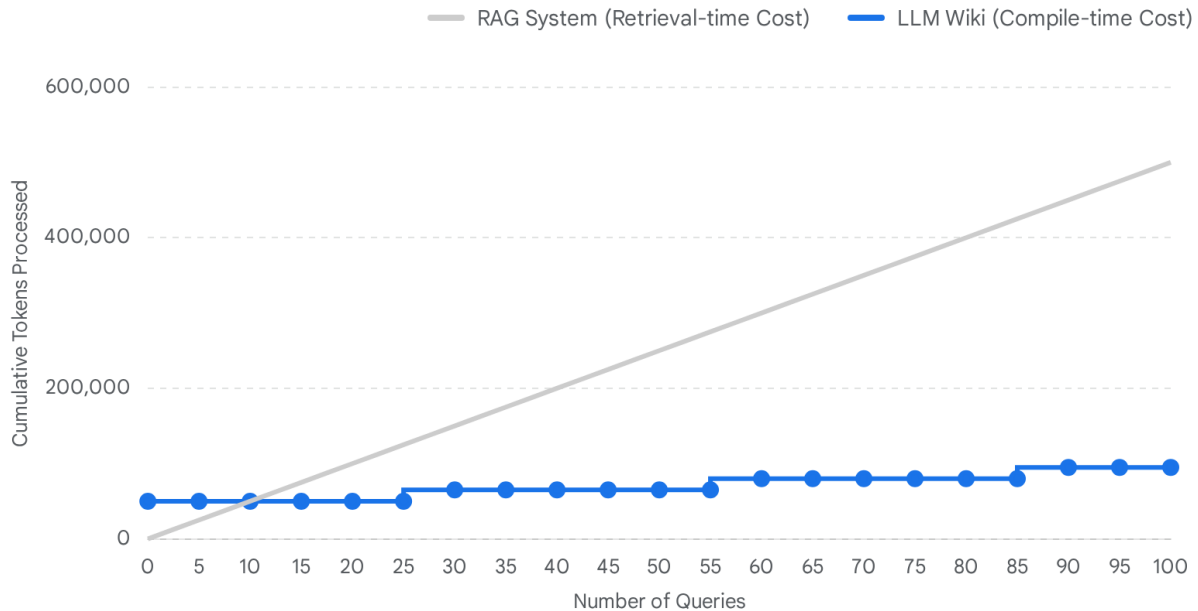


Architecting the Sovereign Epistemic Engine: Compounding Knowledge Systems in Hyperscale, Air-Gapped Environments

The Epistemological Shift in AI Knowledge Management

For years, the dominant architecture for interacting with large corpora of unstructured data has been Retrieval-Augmented Generation (RAG). The standard RAG pipeline operates on a stateless premise: raw documents are chunked and embedded, and at query time, a vector search retrieves the most semantically similar chunks to inject into the Large Language Model (LLM) context window¹. However, advanced implementations and rigorous production testing have exposed a fundamental flaw in this architecture: traditional RAG systems are fundamentally amnesic⁴. Every query forces the system to rediscover knowledge from scratch⁴. If a query requires the synthesis of five distinct documents spanning different time periods to map a subtle relationship, the LLM must independently extract, align, and synthesize those fragments during the high-latency inference phase¹. When the session ends, that synthesis evaporates. The computational effort is discarded, meaning knowledge within the system never compounds, and the user pays the exact same context tax upon the next query⁴. The alternative, popularized by Andrej Karpathy in early 2026, is the "LLM Wiki" or "Second Brain" pattern⁶. This approach fundamentally shifts the computational burden from the query phase to the compilation phase². In this paradigm, the LLM acts as an asynchronous compiler, treating raw text much like a traditional software compiler treats source code². When raw source material—such as an article, transcript, or research paper—enters the system, it is placed in an immutable storage directory⁴. The LLM reads the source, extracts core concepts, updates existing entity pages, notes contradictions, and generates heavily interlinked markdown files in a separate, structured repository⁶. By the time a user queries the system, the heavy lifting of cross-referencing, semantic alignment, and synthesis has already been permanently codified into an evolving markdown knowledge graph⁷. The synthesis is available before the query is even formulated⁵.

Cumulative Computational Load: Retrieval vs. Compilation



The traditional RAG architecture requires re-processing raw context for every query, leading to unsustainable token consumption. The compilation model incurs a higher upfront cost during data ingestion but flattens the query cost curve by storing pre-synthesized knowledge.

Data sources: [AskGlitch](#), [MindStudio](#), [Hjarni](#), [AI Builder Club](#)

The baseline architecture relies on three foundational components to achieve this compounding effect. First is the raw data layer, an immutable directory of source materials (PDFs, web clippings, audio transcripts) that acts as the ground truth⁴. Second is the compiled wiki layer, consisting of LLM-generated markdown pages. These pages utilize an atomic decomposition strategy, categorizing knowledge into discrete entity pages (people, companies, tools) and concept pages (ideas, frameworks)¹. Finally, the schema layer operates as the operational specification, dictating prompt instructions and agent behaviors, effectively turning the LLM into a disciplined wiki maintainer rather than a conversational chatbot⁴. While the Karpathy pattern is highly effective for individual researchers managing thousands of files, it encounters catastrophic scaling barriers in enterprise, hyperscale, and high-security environments. When file systems exceed a critical threshold—typically between 10,000 and 100,000 text files—applications reliant on local file parsing begin to experience severe latency¹⁵. Furthermore, existing automation frameworks are deeply coupled to cloud-hosted APIs⁴. Designing a sovereign, air-gapped knowledge architecture requires taking the philosophical core of the compounding second brain and re-engineering it using specialized,

local-first hyperscale infrastructure.

Assessing the Current Second Brain Ecosystem

The evolution of personal knowledge management (PKM) systems reveals a historical struggle with the human maintenance burden. Applications such as Notion, Roam Research, and Obsidian offered powerful linking capabilities, but relied on the user to manually curate, tag, and cross-reference every insight⁴. This manual bookkeeping inevitably leads to system decay; the cognitive load of organizing data outpaces the value derived from it¹. The LLM Wiki pattern solves this by delegating the bookkeeping to artificial intelligence, allowing the human operator to focus solely on curation and high-level inquiry⁴.

Following the release of the LLM Wiki concept, developers rapidly deployed tools to automate the compilation process. The *llm-wiki-plugin* emerged as a popular extension for Claude Code, bundling slash commands that allow users to ingest documents, lint the knowledge base for contradictions, and query the synthesized data¹³. Similarly, open-source initiatives like *llm-wiki-compiler* and *AtomicMemory* introduced two-phase pipelines that separate the extraction of concepts from the generation of markdown pages, utilizing incremental compile logic that relies on hash-based change detection to process only new or modified sources¹⁰. These tools have demonstrated that multi-agent systems—where diverse coding agents, such as Cursor or local CLI tools, all read and write to the same central markdown repository—can distill experience and correct conventions organically across workflows⁷.

However, scaling these implementations highlights distinct software constraints. Visualizing these networks typically relies on native Obsidian features, which index files using local Electron-based caches¹⁵. While some users report stability at 30,000 to 40,000 files, operations like full-text search, automated refactoring, and dynamic graph generation become computationally expensive and highly sluggish as the vault approaches 100,000 to 200,000 discrete markdown notes¹⁵. Furthermore, systems that rely entirely on basic file system grep operations for retrieval miss the nuanced semantic relationships required for deep reasoning¹³. More critically, these implementations routinely violate the security postures of highly sensitive environments. Frameworks integrating directly with cloud providers transmit proprietary data across external networks, which is an immediate disqualifier for defense, intelligence, and proprietary research sectors²¹. A system designed for these environments must operate 100% offline, process multi-modal inputs locally, construct robust semantic and relational databases that scale to millions of entities, and execute autonomous tasks within secure, tamper-proof sandboxes²².

The Sovereign Citadel Architecture

To resolve the constraints of legacy file-system PKMs and cloud-dependent agents, this report proposes a new hyperscale paradigm: the Citadel Architecture. The Citadel Architecture replaces cloud dependencies and fragile file systems with a robust, local-first software stack designed explicitly for isolated environments. The system processes raw data through local perception models, compiles that data utilizing offline LLMs, and stores the resulting structured

knowledge across a unified address bus that seamlessly connects vector, graph, and file storage components. This entirely air-gapped system eliminates external vulnerabilities while enabling exponential knowledge compounding.

The architecture is structurally divided into four interconnected operational planes: Sensory Ingestion, The Neural Compiler, Tripartite Memory, and The Execution Sandbox. Each plane utilizes specialized, open-source technologies optimized for high throughput, local execution, and strict data sovereignty.

Plane 1: Offline Sensory Ingestion

In an air-gapped environment, data does not arrive cleanly formatted. It enters the system as highly entropic raw material: classified audio recordings, scanned physical documents, multi-page PDFs, and unstructured text²⁴. The first plane of the Citadel Architecture is responsible for local, hardware-accelerated digitization and transcription.

Audio Processing via Local Inference

For speech-to-text requirements, the architecture leverages localized instances of the Whisper model, bypassing API dependencies²⁶. The deployment strategy is bifurcated based on the available hardware infrastructure of the host environment. On Apple Silicon (M-series processors) or low-power embedded systems, whisper.cpp is the optimal engine. Written in pure C/C++, it executes directly on the CPU or via the Apple Metal backend, requiring zero Python infrastructure and utilizing highly optimized SIMD instructions (AVX, AVX2, ARM NEON)²⁶.

Conversely, for environments equipped with NVIDIA GPU clusters—standard in enterprise and defense data centers—the faster-whisper implementation is deployed²⁸. This engine leverages CTranslate2 to enable INT8 quantization, which radically reduces the VRAM required for inference. For example, the Whisper large-v3 model, which encompasses 1.55 billion parameters, requires approximately 10GB of VRAM in standard precision, but operates efficiently under 5GB with INT8 quantization²⁸. This compression yields a negligible degradation in Word Error Rate (WER) while delivering up to a 12x speedup relative to real-time transcription on hardware equivalent to an RTX 4070²⁸.

Model Variant	Optimal Local Engine	Parameter Count	VRAM Requirement (Quantized)	Speed vs. Real-Time	Ideal Hardware Profile
Large-v3	faster-whisper	1.55 Billion	~4.8 GB - 5.0 GB	~12x	NVIDIA RTX 3070 / 4070+ ²⁸

Distil-Large-v3	faster-whisper	~756 Million	~4.0 GB	~18x - 24x	High-throughput NVIDIA clusters ²⁸
Small	whisper.cpp	244 Million	~2.0 GB	~6x - 10x	Apple M-Series / High-end CPU ²⁶
Tiny / Base	whisper.cpp	39M / 74M	~1.0 GB	~16x - 32x	Embedded systems / Raspberry Pi 5 ²⁶

Advanced Document Parsing and Layout Analysis

Transforming dense, unstructured PDFs and images into structured markdown presents a distinct challenge. Traditional OCR engines like Tesseract, while highly mature for standard printed text, struggle significantly with complex layouts, non-standard typography, and handwritten annotations³¹. To achieve maximum accuracy, the Citadel Architecture utilizes open-weight Vision-Language Models (VLMs) such as Qwen2.5-VL or olmOCR²⁵. However, running full-page VLM inference is computationally expensive and prone to hallucination if the page is overly dense²⁵. Therefore, the ingestion plane employs a hybrid pipeline. First, a rapid, localized layout detection model (such as Surya) scans the document to identify distinct bounding boxes for text blocks, tables, and images²⁵. The pipeline then isolates these specific bounding boxes and passes only the cropped sections to the local VLM for transcription²⁵. This localized approach trades a slight increase in orchestration complexity for near-perfect semantic extraction, ensuring that multi-column layouts and embedded charts are properly sequenced into clean markdown before passing to the compiler²⁵.

OCR Methodology	Primary Tooling	Layout Understanding	Semantic / Cursive Parsing	Resource Intensity
Traditional ML	Tesseract / EasyOCR	Moderate	Poor	Low (CPU bound) ³¹
High-Throughput ML	PaddleOCR	High	Moderate	Medium (CPU/GPU) ³¹

VLM (Hybrid Layout)	Surya + olmOCR	Excellent	Excellent	High (GPU bound) ²⁵
VLM (Grounded/Native)	Qwen2.5-VL / MinerU	Excellent	Excellent	Very High (GPU bound) ²⁵

Plane 2: The Neural Compiler

With data safely digitized in the immutable /raw repository, the compilation phase begins. In a cloud-tethered system, this is where external APIs are invoked. In the Citadel Architecture, orchestration is entirely local, utilizing frameworks like Ollama, MLX, or vLLM to serve high-performance open-weight models (e.g., Llama 3.1 8B, Mistral 7B, or Phi-4)²⁰.

The compilation process is not a single, open-ended prompt. Allowing an LLM unstructured read/write access to a knowledge base guarantees eventual degradation through hallucinated relationships or recursive loops¹³. Instead, the system employs a deterministic, two-phase pipeline designed to extract meaning and subsequently align it with existing ontological structures.

In the first phase, the local LLM acts purely as an analytical engine. It reads the new source document and generates a structured JSON output identifying all discrete entities, definitions, claims, and relationships¹⁷. The LLM is explicitly restricted from generating narrative prose during this phase. This mirrors the GraphRAG extraction protocols, where unstructured text is converted into nodes and edges representing relationships²².

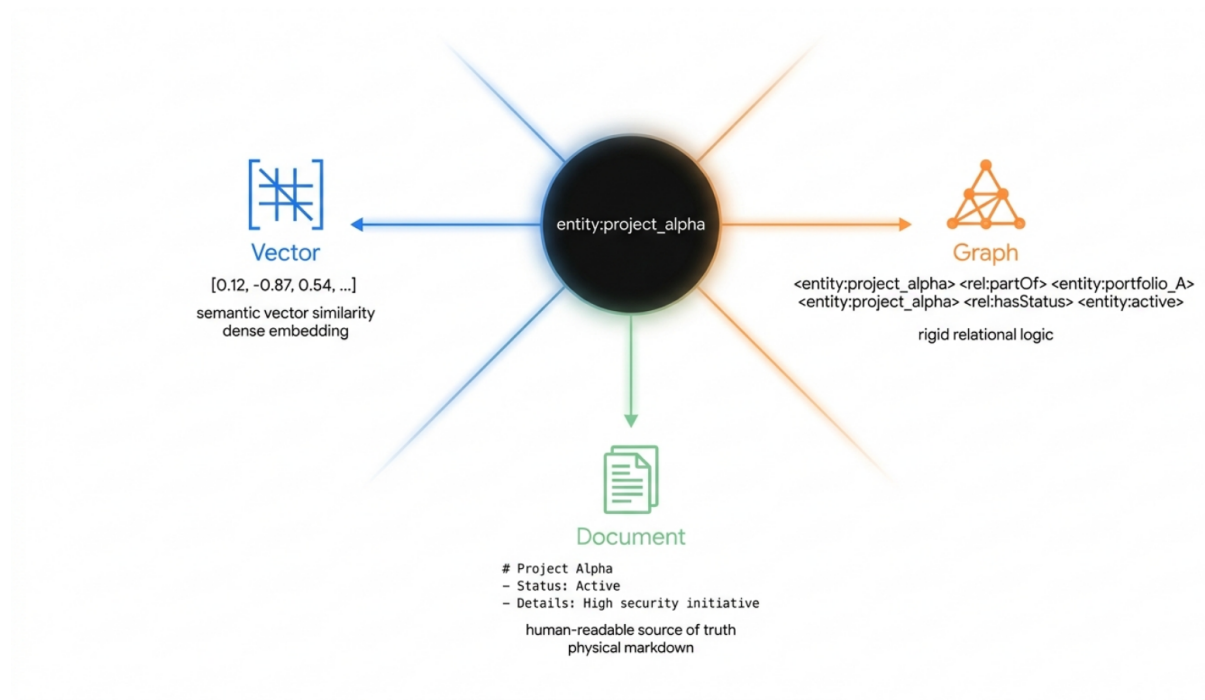
In the second phase, a deterministic compiler script evaluates the extracted entities against the existing database. If an entity or concept is entirely novel, the LLM is prompted to draft a new concept page in markdown¹². If the entity already exists, the system performs surgical, localized updates—such as executing a str_replace operation on specific paragraphs—rather than prompting the model to rewrite the entire document from scratch¹⁶. This strict separation of extraction and mutation ensures that the system fails safely; if the model's confidence threshold falls below a predefined limit, the candidate pages are placed in a review queue rather than merged directly into the master dataset¹⁰. Furthermore, every generated claim is tagged with cryptographic provenance, linking it back to the specific line number in the original raw document, ensuring strict auditability¹⁰.

Plane 3: Tripartite Memory (The Universal Address Bus)

The most significant architectural divergence from the baseline Karpathy pattern occurs at the storage layer. As established, treating a massive directory of markdown files as a primary database creates untenable I/O bottlenecks at hyperscale¹⁵. The Citadel Architecture resolves this by separating the knowledge into three highly specialized, embedded databases. These databases are kept perfectly synchronized through a unified JSON-LD (JavaScript Object Notation for Linked Data) standard, acting as a universal address bus³⁵. Every entity, concept,

and document is assigned a unique Internationalized Resource Identifier (IRI), decoupling the semantic, relational, and lexical storage mechanisms.

Tripartite Memory Unification via JSON-LD



By assigning a unique Internationalized Resource Identifier (IRI) to every entity, the system decouples storage mechanisms. LanceDB handles semantic vector similarity, Oxigraph manages rigid relational logic, and the Markdown file system acts as the human-readable source of truth.

1. Semantic Storage: LanceDB

Vector databases are essential for semantic similarity searches, allowing the system to locate relevant context even when exact keywords differ²³. However, traditional in-memory vector databases (such as FAISS) demand that all high-dimensional vectors reside in RAM, leading to Out-Of-Memory (OOM) failures when scaled to tens of millions of embeddings³⁶. While extensions like `sqlite-vec` provide excellent embedded vector search for small-scale edge deployments or mobile applications, they rely on brute-force L2/Cosine calculations that struggle with latency at million-vector scales³⁶.

For hyperscale environments, the Citadel Architecture utilizes **LanceDB**. Written in Rust, LanceDB is an embedded, serverless vector database that utilizes a zero-copy, columnar storage format directly on disk or object storage³⁹. To circumvent RAM exhaustion, LanceDB employs an Inverted File with Product Quantization (IVF-PQ) index⁴¹. This architecture partitions the dataset and compresses the embeddings, enabling the system to query billions of vectors

while loading only the highly active index segments into memory⁴¹. Furthermore, LanceDB accelerates high-dimensional queries using advanced optimizations like RaBitQ for fast matrix rotation and Hierarchical Navigable Small World (HNSW) routing over partition centroids, bypassing linear centroid scans entirely⁴². This allows LanceDB to execute complex hybrid queries—merging dense vector similarity, full-text search, and SQL metadata filtering—within a single, low-latency operation⁴³.

2. Relational Graph Storage: Oxigraph

While semantic search is powerful for discovering latent relationships, rigorous intelligence analysis requires deterministic, multi-hop logical deductions (e.g., "Identify all subcontractors associated with Entity A who were active in Region B prior to Date C"). Relying on an LLM to hallucinate or probabilistically stitch these connections together at query time is an unacceptable operational risk²². The system requires a rigid knowledge graph.

The architecture embeds **Oxigraph**, a highly performant graph database written in Rust that natively implements the SPARQL querying standard and relies on the robust RocksDB key-value store⁴⁵. (Alternatively, **CozoDB**, a transactional Datalog database, can be used for deep recursive algorithms like PageRank directly within the query language⁴⁷).

The strategic advantage of Oxigraph is its native fluency with JSON-LD³⁵. As the neural compiler extracts entities, it formats them into Subject-Predicate-Object RDF triples, assigning each node a unique IRI³⁵. Oxigraph serves as the logical "hippocampus" of the agentic system. Rather than injecting massive, token-heavy markdown documents into the LLM's context window, the system injects only the relevant IRIs and brief summaries³⁵. When the agent requires deeper context, it executes a SPARQL query against Oxigraph to retrieve precisely targeted relationships, transforming the LLM's context cost from linear $O(n)$ growth to a highly efficient $O(1)$ retrieval mechanism³⁵.

3. Lexical Storage and Virtual Filesystems

To support instantaneous keyword retrieval without the overhead of vector embedding, the architecture incorporates **Tantivy**. Tantivy is an embedded full-text search library written in Rust, engineered as a modern, high-performance evolution of Apache Lucene⁴⁹. Utilizing Finite State Transducers (FSTs) for its term dictionary and SIMD instructions for integer compression, Tantivy delivers sub-millisecond BM25 relevance scoring while operating entirely in-process⁵⁰. Finally, to bridge these highly optimized databases back to the user interface, the system must simulate the traditional "folder of markdown files" expected by tools like Obsidian. Generating millions of physical files on an OS disk invites corruption and indexing latency. Instead, the architecture utilizes **FUSE (Filesystem in Userspace)** via Rust libraries such as fuser and virtual-fs⁵². The system creates a virtual mount point (e.g., /knowledge_base). When the Obsidian frontend attempts to open a markdown file, the FUSE layer intercepts the POSIX request, dynamically queries LanceDB and Oxigraph via the entity's IRI, constructs the markdown text in memory, and streams the bytes back to the application⁵³. The user perceives a seamless folder of physical text files, while the underlying system operates exclusively on

distributed databases, completely eliminating file system bottlenecks.

Storage Layer	Core Technology	Primary Data Structure	Architectural Function
Semantic / Hybrid	LanceDB (Rust)	Columnar / Vectors	IVF-PQ scaling, semantic retrieval, hybrid filtering ³⁹
Relational Graph	Oxigraph / CozoDB (Rust)	RDF Triples / Datalog	Multi-hop deterministic logic, IRI address bus ³⁵
Lexical Search	Tantivy (Rust)	Inverted Index / FSTs	Sub-millisecond BM25 keyword matching ⁵⁰
Virtual Filesystem	FUSE (fuser)	In-Memory POSIX	Dynamic markdown generation for UX rendering ⁵³

Plane 4: Zero-Trust Security and Sandboxing

In a heavily fortified, air-gapped environment, external network attacks are neutralized. The primary vector for compromise shifts to internal privilege escalation. When an autonomous LLM is granted the authority to execute scripts, compile databases, and organize files, it becomes uniquely susceptible to prompt injection attacks embedded within the raw source material⁵⁶. If a malicious PDF contains a hidden instruction directing the agent to truncate the primary database, a naive architecture will dutifully execute the command.

WebAssembly (WASM) Isolation

To mitigate this, the Citadel Architecture strictly sandboxes all agent-driven tools and execution scripts using **WebAssembly (WASM)**⁵⁶. WASM provides a highly restricted, stack-based virtual machine environment⁵⁸. Unlike Docker containers, which share the host operating system's kernel and introduce significant computational overhead, WASM modules execute in isolated linear memory with zero innate access to the host file system, network interfaces, or environment variables⁵⁷. Access is governed by a strict, capability-based security model; if a capability is not explicitly granted by the host runtime, the module cannot access it⁵⁸. This security paradigm is heavily utilized in high-performance proxy servers like Envoy and Istio, where custom extension plugins are compiled to WASM to ensure that a crashing or malicious plugin cannot compromise the primary proxy infrastructure⁵⁹. By adapting this model, the Citadel Architecture ensures that every compilation script or retrieval function invoked by the

local LLM runs within a disposable WASM sandbox. Any attempt to escalate privileges or execute unauthorized system calls is instantly terminated by the runtime⁵⁸.

Cryptographic Secret Management

Even in completely offline environments, the orchestration system requires access to sensitive credentials, such as database encryption keys or local API tokens. Storing these credentials in plaintext configuration files (such as .env or CLAUDE.md) is a severe security vulnerability⁴. The architecture manages credentials utilizing **Agenix**, a cryptographic tool designed for NixOS environments that leverages existing SSH public/private key infrastructure⁶¹. Instead of managing plaintext strings, administrators encrypt secrets into .age files and commit them directly into the configuration repository alongside the application logic⁶¹. During system boot or application runtime, the host machine uses its private SSH key to automatically decrypt the .age files and mount them into a volatile, memory-backed filesystem (typically /run/agenix/)⁶¹. The decrypted secrets exist only in RAM and never touch the physical disk⁶². If the physical hardware is powered down or seized by unauthorized personnel, the encrypted data remains completely inaccessible without the ephemeral keys, ensuring total data sovereignty at rest.

Synthesized Operational Workflows

The confluence of these four specialized planes enables a highly orchestrated, automated lifecycle for raw intelligence entering the system.

1. **Secure Ingestion:** An operator introduces an encrypted dataset (audio files and scanned PDFs) into the ingestion staging area. Agenix seamlessly decrypts the payload into volatile memory⁶⁴. The local perception layer triggers: faster-whisper handles the audio processing, while the Surya/VLM hybrid pipeline parses the complex documents²⁹. The output is pristine, unformatted markdown resting in the immutable /raw repository.
2. **Autonomous Compilation:** The system detects the new files and initiates the Two-Phase Neural Compiler. An offline LLM (e.g., Llama 3) processes the raw text, extracting a highly structured JSON mapping of all entities, claims, and contradictions¹⁷. The model is constrained from overwriting data directly; it merely generates extraction blueprints.
3. **Tripartite Synchronization:** The extraction scripts, running securely within WASM sandboxes, parse the LLM's blueprints. A local embedding model generates high-dimensional vectors for the text and commits them to LanceDB's columnar storage⁴⁰. Simultaneously, the exact relational logic (e.g., Entity_A --associated_with--> Entity_B) is codified into RDF triples and committed to the Oxigraph database⁴⁶. All data is bonded together by unique JSON-LD IRIs³⁵.
4. **Retrieval and Synthesis:** A human analyst opens their local markdown editor (e.g., Obsidian). When they search for a concept, the FUSE virtual filesystem dynamically constructs the markdown page on the fly by querying LanceDB for semantic context, Tantivy for exact keyword matches, and Oxigraph for the relational graph⁴². If the analyst asks the local LLM agent a complex question, the agent queries the Oxigraph "hippocampus" to pull only the strictly necessary IRIs, synthesizing a hallucination-free,

perfectly cited response using minimal context window tokens³⁵.

Conclusion

The emergence of the LLM Wiki and the concept of a compounding "Second Brain" marked a pivotal evolution in artificial intelligence. It demonstrated that shifting the computational burden from query-time retrieval to ingest-time compilation yields vastly superior long-term knowledge retention and synthesis. However, the initial implementations—relying heavily on flat file systems and cloud-based APIs—are structurally incapable of surviving the rigorous demands of hyperscale, high-security enterprise environments.

The Citadel Architecture proposed in this report bridges that gap. By replacing fragile file-system operations with specialized, embedded infrastructure—LanceDB for zero-copy vector scaling, Oxigraph for SPARQL-driven relational logic, and Tantivy for microsecond lexical retrieval—the system can gracefully scale to process millions of documents without degradation. Securing this pipeline with WebAssembly sandboxing and Agenix cryptography guarantees that this autonomous knowledge generation can occur safely within the most restrictive, air-gapped environments. The result is a sovereign epistemic engine: an autonomous system that continuously extracts, maps, and safeguards intelligence, transforming scattered raw data into a permanent, highly queryable strategic asset.

Works cited

1. The Second Brain That Grows Smarter and Lives on your Computer | by Dr. Tali Rezun,
<https://medium.com/@talirezun/the-second-brain-that-grows-smarter-and-lives-on-your-computer-59dd45031db4>
2. What Is Andrej Karpathy's LLM Knowledge Base Architecture? The Compiler Analogy Explained | MindStudio,
<https://www.mindstudio.ai/blog/karpathy-llm-knowledge-base-architecture-compiler-analogy>
3. GraphRAG: Hierarchical Approach to Retrieval-Augmented Generation - LanceDB,
<https://www.lancedb.com/blog/graphrag-hierarchical-approach-to-retrieval-augmented-generation>
4. Build a Second Brain: Karpathy's LLM Wiki Method, Step by Step | Professor Glitch,
<https://www.askglitch.com/blog/build-a-second-brain>
5. Karpathy's LLM Knowledge Base Method - A Practical Starting Point | Jamal Hansen, <https://jamalhansen.com/blog/road-to-agentic-notes/>
6. LLM Wiki by Andrej Karpathy: Build a Compounding Knowledge Base (Tutorial), <https://datasciencedojo.com/blog/llm-wiki-tutorial/>
7. llm-wiki · GitHub,
https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f?permalink_comment_id=6083543
8. Andrej Karpathy's LLM Wiki: Build a Personal Knowledge Base with Obsidian and Codeex in 5 Minutes | MindStudio,

- <https://www.mindstudio.ai/blog/andrej-karpathy-llm-wiki-obsidian-codeex-second-brain>
9. Karpathy's LLM Wiki: A Knowledge Base That Compounds - AI Builder Club,
<https://www.aibuilderclub.com/blog/karpathy-llm-wiki>
 10. GitHub - atomicstrata/llm-wiki-compiler: The knowledge compiler. Raw sources in, interlinked wiki out. Inspired by Karpathy's LLM Wiki pattern.,
<https://github.com/atomicstrata/llm-wiki-compiler>
 11. Andrej Karpathy's LLM Wiki: Create your own knowledge base | by Urvil Joshi | Medium,
<https://medium.com/@urvvil08/andrej-karpathys-llm-wiki-create-your-own-knowledge-base-8779014accd5>
 12. I Stopped Taking Notes and Built a Second Brain That Maintains Itself | by Deyves Senger,
<https://medium.com/@deyves.senger/i-stopped-taking-notes-and-built-a-second-brain-that-maintains-itself-80075b7643fb>
 13. LLM Wiki — Compounding Knowledge Base for Claude Code - GitHub,
<https://github.com/6eanut/llm-wiki>
 14. Karpathy's LLM Wiki? No Code with Claude or Github Copilot! - DEV Community,
<https://dev.to/rosidotidev/karpathys-llm-wiki-no-code-with-claude-or-github-copilot-5fb0>
 15. Can obsidian support many files without lagging ? : r/ObsidianMD - Reddit,
https://www.reddit.com/r/ObsidianMD/comments/1qgz78z/can_obsidian_support_many_files_without_lagging/
 16. Andrej Karpathy's LLM Wiki pattern as a Claude Code plugin - GitHub,
<https://github.com/praneybehl/llm-wiki-plugin>
 17. GitHub - atomicmemory/llm-wiki-compiler at genaisecretsauce.com,
<https://github.com/atomicmemory/llm-wiki-compiler?ref=genaisecretsauce.com>
 18. Indexing Large Number of Files - Bug graveyard - Obsidian Forum,
<https://forum.obsidian.md/t/indexing-large-number-of-files/47397>
 19. Maximum Number of Notes in Vault - Page 2 - Help - Obsidian Forum,
<https://forum.obsidian.md/t/maximum-number-of-notes-in-vault/1509?page=2>
 20. Spent months building a fully offline RAG + knowledge graph app for Mac. Everything runs on-device with MLX. Here's what I learned. : r/LocalLLaMA - Reddit,
https://www.reddit.com/r/LocalLLaMA/comments/1rea7fb/spent_months_building_a_fully_offline_rag/
 21. What's the deal with the hype around Karpathy's LLM wiki? : r/ObsidianMD - Reddit,
https://www.reddit.com/r/ObsidianMD/comments/1sx040s/whats_the_deal_with_the_hype_around_karpathys_llm/
 22. GraphRAG on Consumer Hardware: Benchmarking Local LLMs for Healthcare EHR Schema Retrieval - arXiv, <https://arxiv.org/html/2605.20815v1>
 23. Why your digital library system needs a knowledge graph: building offline document intelligence - Carlo C.,
<https://autognosi.medium.com/why-your-digital-library-system-needs-a-knowledge>

[dge-graph-building-offline-document-intelligence-ed856c8f4195](#)

24. Andrej Karpathy's LLM Knowledge Bases explained | by Mehul Gupta - Medium, <https://medium.com/data-science-in-your-pocket/andrej-karpathys-llm-knowledge-bases-explained-2d9fd3435707>
25. GitHub - ahnafnafee/local-llm-pdf-ocr: Convert scanned PDFs into searchable text locally using Vision LLMs (olmOCR). 100% private, offline, and free. Features a modern Web UI & CLI., <https://github.com/ahnafnafee/local-llm-pdf-ocr>
26. Whisper.cpp GUI for Windows, CUDA + Vulkan + CPU, No Terminal - StarWhisper, <https://starwhisper.ai/landing/whisper-cpp-windows.html>
27. Whisper.cpp Setup Guide: Run Speech Recognition Locally - Weesper Neon Flow, <https://weesperneonflow.ai/en/blog/2026-06-23-whisper-cpp-setup-guide-local-speech-recognition-2026/>
28. Whisper.cpp vs faster-whisper 2026: Local STT Benchmarks, Setup & GPU Acceleration, <https://www.promptquorum.com/power-local-llm/local-whisper-stt-comparison-2026>
29. Self-Hosted Whisper in 2026: Local AI Transcription Guide - Digital Applied, <https://www.digitalapplied.com/blog/local-speech-to-text-whisper-self-hosted-transcription-2026>
30. Run Whisper Locally 2026: Free Offline Speech-to-Text Setup | Local AI Master, <https://localaimaster.com/blog/whisper-local-speech-to-text>
31. Top 7 Open source OCR tools you need to know - Mindee, <https://www.mindee.com/blog/top-open-source-ocr-tools>
32. Building a Local LLM-Powered Hybrid OCR Engine - Ahnaf An Nafee, <https://ahnafnafee.dev/blog/local-llm-pdf-ocr>
33. Best Open Source OCR Tools & Models for Developers in 2026 - Unstract, <https://unstract.com/blog/best-opensource-ocr-tools/>
34. GraphRAG Architecture: Components, Workflow & Implementation Guide - PuppyGraph, <https://www.puppygraph.com/blog/graphrag-architecture>
35. Gliding Horse — I Chose Oxigraph as My AI's Brain, and the Whole System Went Beast Mode | by doiito | CodeToDeploy | May, 2026 | Medium, <https://medium.com/codetodeploy/gliding-horse-i-chose-oxigraph-as-my-ais-brain-and-the-whole-system-went-beast-mode-8792183cccc9>
36. How sqlite-vec Works for Storing and Querying Vector Embeddings | by Stephen Collins, <https://medium.com/@stephenc211/how-sqlite-vec-works-for-storing-and-querying-vector-embeddings-165adeeeceea>
37. Embedded Vector Databases for Go in 2026: chromem-go vs sqlite-vec vs Bleve vs LanceDB - Shaharia Azam, <https://shaharia.com/blog/choosing-embeddable-vector-database-go-application/>
38. Glia – Local-first shared memory layer (SQLite-vec + FTS5 + Offline Knowledge Graph), https://www.reddit.com/r/KnowledgeGraph/comments/1tggapm/glia_localfirst_shared_memory_layer_sqlitevec/

39. Best Vector Databases in 2026: Complete Comparison Guide - Encore Cloud, <https://encore.dev/articles/best-vector-databases>
40. Vector search on object storage: Performance at scale without the infrastructure tax - LanceDB, <https://lancedb.com/lp/vector-db-guide/>
41. LanceDB vs Deep Lake on Vector Search Capabilities - Zilliz blog, <https://zilliz.com/blog/lance-db-vs-deep-lake-a-comprehensive-vector-database-comparison>
42. How LanceDB Accelerates Vector Search at 10 Billion Scale, <https://www.lancedb.com/blog/how-lancedb-accelerates-vector-search-at-10-billion-scale>
43. LanceDB | Multimodal Lakehouse for AI, <https://www.lancedb.com/>
44. Hybrid Search and Custom Reranking with LanceDB, <https://www.lancedb.com/blog/hybrid-search-and-custom-reranking-with-lancedb-4c10a6a3447e>
45. Oxigraph - graph database - LinuxLinks, <https://www.linuxlinks.com/oxigraph-graph-database/>
46. Architecture · oxigraph/oxigraph Wiki - GitHub, <https://github.com/oxigraph/oxigraph/wiki/Architecture>
47. GitHub - lawless-m/cozo-redb: A faithful ledger of facts, their kinships, and their nearest neighbours — interrogated in the plain and honest tongue of Datalog., <https://github.com/lawless-m/cozo-redb>
48. GitHub - cozodb/cozo: A transactional, relational-graph-vector database that uses Datalog for query. The hippocampus for AI!, <https://github.com/cozodb/cozo>
49. Introduction to Tantivy - ParadeDB, <https://www.paradedb.com/learn/tantivy/introduction>
50. Tantivy is a full-text search engine library inspired by Apache Lucene and written in Rust - GitHub, <https://github.com/quickwit-oss/tantivy>
51. Of tantivy, a search engine in Rust - Paul Masurel, <https://fulmicoton.com/posts/ behold-tantivy/>
52. virtual-fs - crates.io: Rust Package Registry, <https://crates.io/crates/virtual-fs>
53. cberner/fuser: Filesystem in Userspace (FUSE) for Rust - GitHub, <https://github.com/cberner/fuser>
54. Building a tiny FUSE filesystem - Shayon Mukherjee, <https://www.shayon.dev/post/2026/161/building-a-tiny-fuse-filesystem/>
55. How to create a virtual filesystem in Rust? - Stack Overflow, <https://stackoverflow.com/questions/40057303/how-to-create-a-virtual-filesystem-in-rust>
56. Introduction - IronClaw, <https://logicminds-ironclaw.mintlify.app/introduction>
57. Browser Sandboxing for Coding Agents: 2026 Security Guide | Blaxel Blog, <https://blaxel.ai/blog/browser-sandboxing-for-coding-agents>
58. Securing the Stack: How to Build Zero-Trust Applications with WebAssembly, <https://alain-airom.medium.com/securing-the-stack-how-to-build-zero-trust-applications-with-webassembly-1492f35284de>
59. How to Extend Istio with WebAssembly (WASM) Plugins - OneUptime,

- <https://oneuptime.com/blog/post/2026-01-07-istio-wasm-plugins/view>
60. API Gateway Plugin Architectures Compared (2026): Lua, Go, WASM, and TypeScript,
<https://zuplo.com/learning-center/api-gateway-plugin-architectures-compared>
61. ryanm/agenix: age-encrypted secrets for NixOS and Home manager - GitHub,
<https://github.com/ryanm/agenix>
62. Agenix - NixOS Wiki, <https://nixos.wiki/wiki/Agenix>
63. NixOS with agenix | Woile, <https://woile.dev/blog/agenix.html>
64. Nix secrets for dummies | Farid Zakaria's Blog,
<https://fzakaria.com/2024/07/12/nix-secrets-for-dummies>